

Programmation objet en Java - Feuille de TP 4

À partir de ce TP, nous utiliserons exclusivement l'IDE (environnement de développement) *Eclipse*. Ce logiciel propose plusieurs fonctionnalités simplifiant le travail en Java : compilation, gestion des packages, détection de certaines erreurs avant compilation, etc.

Exercice 1. Vérification de l'environnement Eclipse

Eclipse est normalement déjà configuré sur les machines de l'université. L'objectif de cet exercice est de vérifier que cette configuration est correcte.

1. Lancez `eclipse light`.
2. Commencez par choisir comme *workspace* le dossier dans lequel vous stockez vos TPs de Java. Pour cela, allez dans *File* → *Switch Workspace*.
3. Dans le menu *Window* → *Preferences* → *Java* → *Installed JREs*, vérifiez que l'environnement d'exécution Java utilisé est bien **Java-25**.
 - Si plusieurs JRE sont présents, assurez-vous que **Java-25** (ou **jdk-25**) est sélectionné.
 - S'il n'apparaît pas, demandez de l'aide à un enseignant.
4. **Question :** Qu'est-ce qu'un JRE ? Quelle est la différence entre JRE et JDK ? Faites une recherche, et/ou demandez à une I.A. de générer une explication simple à comprendre à votre niveau.
5. Dans le menu *Window* → *Preferences* → *Java* → *Compiler*, vérifiez que le niveau du compilateur (*Compiler compliance level*) est également réglé sur **Java-25**.

Exercice 2. Création d'un projet Java et gestion des packages

Dans cet exercice, nous allons créer un projet Java dans Eclipse, importer du code existant, puis comprendre comment les *packages* influencent l'organisation des fichiers et la compilation.

1. Allez dans *File* → *New* → *Java Project* pour créer un nouveau *Java Project* nommé **TP-Eclipse**. Dans la fenêtre de création du projet, **décochez la case** *Create module-info.java* située en bas de la fenêtre, puis validez avec *Finish*. Qu'observez-vous dans votre répertoire de travail ?
2. Vérifiez que le projet **TP-Eclipse** apparaît bien dans le panneau de gauche. À l'intérieur du projet se trouve un dossier nommé **src**. C'est dans ce dossier que se trouvent les fichiers source Java.
3. Importez dans le dossier **src** les fichiers du TP précédent : **Book.java** et **BookStore.java**. Pour cela, faites un clic droit sur **src** puis choisissez *Import* → *General* → *File System*. Naviguez ensuite vers le dossier dans où se trouvent vos fichiers du TP précédent à l'aide de *Browse*, puis sélectionnez-le (*Il est normal qu'aucun fichier n'apparaisse avant d'avoir sélectionné le dossier*). Cochez ensuite les fichiers à importer, puis *Finish*.
4. Les fichiers importés se trouvent maintenant dans un *package* (probablement le *default package*). **Il ne faut pas laisser de code dans le package par défaut.**
Créez un nouveau package nommé **fr.but.info.book** (en faisant un clic droit sur **src**, puis *New* → *Package*).
5. **Question :** Qu'est-ce qu'un package, et quelle est son utilité ? Faites une recherche, et/ou demandez à une I.A. de générer une explication simple à comprendre à votre niveau.

6. Déplacez les fichiers `Book.java` et `BookStore.java` dans le package `fr.but.info.book`. Pour cela, sélectionnez les fichiers, faites un clic droit puis choisissez *Refactor* → *Move*. Vous pouvez supprimer le *default package* s'il existe encore.
7. Ouvrez le fichier `Book.java`. Observez la première ligne du fichier. Comparez-la avec l'organisation des dossiers dans votre explorateur de fichiers.

Remarque : en Java, la structure des dossiers correspond exactement au nom du package.
8. Créez maintenant un second package nommé `fr.but.info.book.main`. Ce package contiendra les classes servant uniquement à tester votre code.
9. Importez dans ce package le fichier `BookStoreTest.java` que vous aviez écrit lors du TP précédent. Faites attention à l'endroit où vous effectuez le clic droit lors de l'import : faites le sur `src`, et pas sur votre package.
10. Ouvrez le fichier `BookStoreTest.java`. Une erreur apparaît probablement au niveau de la déclaration de la classe.
11. **Question.** En passant la souris sur l'erreur, Eclipse vous propose une ou plusieurs corrections. Appliquez la correction appropriée et expliquez brièvement ce qui a été modifié.
12. D'autres erreurs peuvent apparaître car la classe `BookStoreTest` se trouve dans un package différent de `Book` et `BookStore`. Utilisez les propositions d'Eclipse pour importer les classes nécessaires.
13. Lorsque toutes les erreurs sont corrigées, exécutez le programme à l'aide du bouton *Run*. Repérez dans votre projet l'emplacement des fichiers `.class` générés.
14. (**Outil Eclipse : commenter du code.**) Dans Eclipse, vous pouvez commenter/décommenter rapidement une ou plusieurs lignes de code : sélectionnez des lignes puis utilisez le raccourci `Ctrl + Shift + C`.

Préparation pour l'exercice suivant. Dans l'exercice 3, la classe `BookStore` va changer : elle ne contiendra plus uniquement des `Book`. Certaines méthodes du `BookStore` ne seront donc plus compatibles immédiatement. Ces méthodes seront réécrites ou adaptées dans la suite du TP. Les commenter permet d'avancer étape par étape sans erreurs de compilation.

Dans `BookStore.java`, commentez temporairement les méthodes suivantes :

- `add(Book b)`
- `booksByAuthor(String author)`
- `authorBeginsWith(String prefix)`
- `contains(Book b)`

Exercice 3. La librairie ne vend plus uniquement des livres, mais également des *comics*. Nous souhaitons intégrer ces deux types d'objets dans un même catalogue.

Objectifs :

- ajouter une nouvelle classe `Comic` ;
- comprendre pourquoi on a besoin d'une abstraction commune ;
- introduire une interface parce qu'un besoin apparaît ;
- manipuler une collection contenant des objets de types différents.

1. Créez une nouvelle classe `Comic` dans le package `fr.but.info.book`. Un comic possède les champs suivants :
 - `title` (le titre) ;
 - `writer` (le scénariste) ;
 - `artist` (la dessinatrice) ;
 - `series` (la série) ;
 - `price` (le prix).
2. Utilisez les touches `Ctrl + Space` dans la classe `Comic`. Créez le constructeur canonique à cinq arguments; cherchez dans les menus si Eclipse ne peut pas vous aider à le faire.
3. Réutilisez `Ctrl + Space` pour vous aider à écrire la méthode d’affichage.
4. Dans votre classe de test (par exemple `BookStoreTest` dans `fr.but.info.book.main`), créez quelques `Comic` et affichez-les. (voir l’exemple ci-dessous) Vérifiez que tout compile et que l’affichage est satisfaisant.

```
static void main(){
    ...
    var c1 = new Comic(
        "Batman: Year One",
        "Frank Miller",
        "David Mazzucchelli",
        "Batman",
        12.5
    );
    var c2 = new Comic(
        "Spider-Man: Blue",
        "Jeph Loeb",
        "Tim Sale",
        "Spider-Man",
        13.0
    );
    IO.println(c1);
    IO.println(c2);
    ...
}
```

5. **Question.** Dans l’état actuel de votre code, est-il possible de stocker à la fois des `Book` et des `Comic` dans le catalogue du `BookStore` ? Justifiez brièvement votre réponse.
6. Pour résoudre le problème identifié à la question précédente, nous allons introduire une interface représentant un article vendu par la librairie.
Créez une interface `CatalogItem` dans le package `fr.but.info.book`.
7. Modifiez `Book` et `Comic` pour qu’elles implémentent `CatalogItem`.
8. Modifiez la classe `BookStore` pour qu’elle représente désormais un catalogue d’articles.
9. Dans `BookStore`, adaptez la méthode `add` (normalement commentée) afin de pouvoir ajouter aussi bien des `Book` que des `Comic` dans le catalogue.
10. Utilisez le code de test ci-dessous pour vérifier que votre `BookStore` peut maintenant contenir à la fois des livres et des comics.

```
static void main(String[] args) {
    var store = new BookStore();

    var b1 = new Book("Lolita", "Nabokov", 20.0);
    var b2 = new Book("Les fruits du Congo", "Vialatte", 20.0);

    var c1 = new Comic(
```

```

        "Batman: Year One",
        "Frank Miller",
        "David Mazzucchelli",
        "Batman",
        12.5
    );

    store.add(b1);
    store.add(c1);
    store.add(b2);

    IO.println("store:");
    IO.println(store);
}

```

Vérifiez que le programme compile et que le catalogue contient bien des livres et des comics.

11. Adaptez et décommentez la méthode `contains` dans `BookStore` afin de vérifier si un article est présent dans le catalogue. La méthode doit fonctionner aussi bien pour les livres que pour les comics.
12. Dans la classe `BookStore`, on souhaite pouvoir calculer le prix total de tous les articles du catalogue. Est-ce possible dans l'état actuel de l'interface `CatalogItem` ? Expliquez brièvement.
13. Modifiez l'interface `CatalogItem` afin de permettre au `BookStore` de calculer le prix total du catalogue.
14. Utilisez le code de test ci-dessous. Il suppose que vos constructeurs existent et que `Book` et `Comic` implémentent bien `CatalogItem`. **Vous pouvez adapter les titres et les prix, mais gardez l'idée** : une librairie contient à la fois des livres et des comics, puis on affiche le catalogue et le total.

```

static void main(String[] args) {
    var store = new BookStore();

    var b1 = new Book("Lolita", "Nabokov", 20.0);
    var b2 = new Book("Les fruits du Congo", "Vialatte", 20.0);

    var c1 = new Comic(
        "Batman: Year One",
        "Frank Miller",
        "David Mazzucchelli",
        "Batman",
        12.5
    );
    var c2 = new Comic(
        "Spider-Man: Blue",
        "Jeph Loeb",
        "Tim Sale",
        "Spider-Man",
        13.0
    );

    store.add(b1);
    store.add(b2);
    store.add(c1);
    store.add(c2);

    IO.println("store:");
    IO.println(store);

    IO.println("total price: " + store.totalPrice());
}

```

15. On souhaite maintenant améliorer l’affichage du `BookStore`. La méthode `toString()` doit afficher **un article par ligne**, sans ligne vide à la fin.

Par exemple, on souhaite obtenir :

```
store:
[Book] ...
[Comic] ...
[Book] ...
```

16. Réécrivez la méthode `toString()` de `BookStore` en utilisant un `StringBuilder`. La méthode doit parcourir les articles du catalogue et ajouter chacun d’eux au résultat. *Indication* : une façon simple d’éviter un retour à la ligne final est de n’ajouter “\n” **qu’avant** chaque élément sauf le premier.

Indication : vous pouvez partir du squelette suivant :

```
@Override
public String toString() {
    var sb = new StringBuilder();
    boolean first = true;
    for (var item : this.catalog) {
        if (!first) sb.append("\n");
        sb.append(item);
        first = false;
    }
    return sb.toString();
}
```

Testez votre affichage avec le code suivant :

```
IO.println("store:");
IO.println(store);
```

17. **Bonus. (Recherche par créateurice)** Dans le TP précédent, `BookStore` contenait uniquement des livres et il était possible de rechercher des livres par auteur.

Dans la version actuelle, le catalogue contient à la fois des `Book` et des `Comic`. Pour pouvoir proposer une recherche similaire sans distinguer les types, nous allons introduire une notion plus générale : la ou les personnes qui ont créé l’article.

- Ajoutez dans l’interface `CatalogItem` une méthode `String creator();`
- Implémentez cette méthode :
 - dans `Book`, `creator()` renvoie l’auteur ;
 - dans `Comic`, `creator()` renvoie une chaîne de caractères contenant le scénariste et la dessinatrice (par exemple “writer / artist”).
- Dans `BookStore`, écrivez une méthode `List<CatalogItem> itemsByCreator(String creator)` qui renvoie la liste des articles dont le résultat de `creator()` contient la chaîne passée en paramètre.
- Testez votre méthode avec le code suivant :

```
static void main(String[] args) {
    var store = new BookStore();

    store.add(new Book("Lolita", "Nabokov", 20.0));
    store.add(new Book("Pnin", "Nabokov", 18.0));

    store.add(new Comic(
        "Batman: Year One",
        "Frank Miller",
        "David Mazzucchelli",
        "Batman",
        12.5
    ));
}
```

```
store.add(new Comic(  
    "Daredevil: Born Again",  
    "Frank Miller",  
    "David Mazzucchelli",  
    "Daredevil",  
    13.0  
));  
  
IO.println("items by Nabokov:");  
IO.println(store.itemsByCreator("Nabokov"));  
  
IO.println("items by Frank Miller:");  
IO.println(store.itemsByCreator("Frank Miller"));  
  
IO.println("items by Mazzucchelli:");  
IO.println(store.itemsByCreator("Mazzucchelli"));  
}
```

Exercice 4. Bonus - Si vous avez tout fini
Finissez les exercices du 3e CM (à partir de la p.45).